8B LLM의 도구 사용 실패 분석: 형식 준수, 행동 선택 및 실행 종료

신다윗*, 안드로*, 아리프*, 조재한*, 이창율*, 하업준*, 흐엉**, 김호원**

*부산대학교 (대학원생), **부산대학교 (교수)

Failure Analysis of Tool Use in 8B LLMs: Format Compliance, Action Selection, and Execution Termination

Dawit Shin*, Andro Aprila Adiputra*, Alif Wicaksana Ramadhan*, Jaehan Cho*, Changyul Lee*, Upjun Ha*, Thi Thu Huong Le**, Howon Kim**

*Pusan National University(Graduate student),

**Pusan National University(Professor)

요 약

LLM의 도구 사용 성능은 일반적으로 최종 성공률로 평가되지만, 하나의 최종 실패에는 형식 오류, 잘못된 행동 선택, 최종 응답 미제출 등 서로 다른 실패가 포함될 수 있다. 본 연구는 8B급 LLM을 대상으로 이러한 실패를 형식 준수, 행동 선택, 실행 종료의 지점으로 구분한다. When2Call 40문항을 각 8회 평가한 결과, 형식 준수율은 96.3~99.1%였으나 행동 선택 정확도는 41.3~65.0%였다. Loc-Bench Security 28문항을 각 8회 평가한 결과, Granite의 format_stop은 86.6%였고 진단용 단일 파서 복구 규칙 적용 후 51.8%로 감소했으며, 같은 조건에서 tool_cap은 10.7%에서 38.8%로 증가하였다. 이러한 결과는 8B 환경에서 최종 정확도와 함께 실패 지점을 분석할 필요가 있음을 보여준다.

I. 서론

LLM 기반 에이전트는 모델과 프롬프트, 도구 인터페이스, 상태 관리 및 실행 제어를 담당하는 에이전트 실행 계층(harness)의 결합으로 구성된다. 이러한 구조는 프론티어급 모델뿐 아니라 로컬 환경에서 실행 가능한 모델에도 적용할 수 있다. 본 연구는 8B급 LLM의 도구 사용 과정에서 관찰되는 실패를 분석한다.

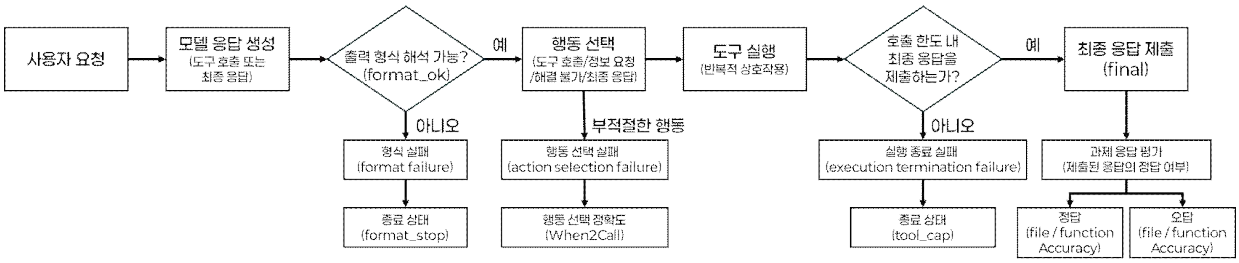
Berkeley Function Calling Leaderboard(BFCL)는 단일·다중 함수 호출과 상태를 갖는 다단계 환경에서 함수 호출 능력을 평가하고[1], When2Call은 BFCL에서 유래한 문항을 이용해 도구 호출, 추가 정보 요청, 해결 불가 판단을 구분한다[2]. 이들 벤치마크는 LLM의 함수 호출과 행동 선택 능력을 정량화한다.

실제 평가에서는 형식 해석 실패, 부적절한 행동 선택, 호출 한도 내 최종 응답 미제출, 제출 후 오답이 모두 최종 실패로 집계될 수 있다. 따라서 최종 정확도만으로 주요 병목을 구분하기 어렵다. CyberGym은 실제 코드베이스 탐색과 실행 피드백을 통한 반복적 취약점 분석을 평가한다[3]. 이러한 보안 에이전트 과제에서도 모델의 판단뿐 아니라 출력 형식, 도구 인터페이스, 반복 실행 및 종료 과정의 실패를 구분할 필요가 있다.

최근 연구는 도구 인터페이스, 상태 관리 및 복구 로직 등 실행 계층의 구성이 에이전트 성능에 영향을 줄 수 있음을 보고한다[4-6]. 본 연구는 8B 환경에서 형식 준수, 행동 선택, 실행 종료를 구분하고, 특정 형식 오류를 완화한 진단 조건에서 후속 종료 상태의 변화를 관찰한다. 이를 위해 다음 연구 질문을 설정한다.

RQ1. 높은 형식 준수율에서도 행동 선택 오류가 남는가?

RQ2. 형식 관련 중단을 줄이면 반복 실행의 종료 상태는 어떻게 변하는가?



[그림 1] LLM의 도구 사용 실행 과정에서 관찰되는 실패 지점과 평가 지표

II. 실험 설계

표 1은 두 평가 축과 공통 실행 환경을 요약한다. 모델별 기본 추론 프로필은 Granite thinking on/temperature 1.0, Ministral thinking off/temperature 0.7로 설정하였다. 이 차이로 인해 모델 간 수치 차이를 순수한 모델 효과로 해석하지 않는다.

[표 1] 실험 환경 및 평가 구성 요약

항목	When2Call	Loc-Bench Security
분석 대상	형식 준수·행동선택	실행 종료·최종 정답
평가 문항	40문항 × 8회	28문항 × 8회
모델	Granite 4.2-8B, Ministral-8B-Instruct	
출력 형식	3-action JSON;	strict JSON;
추가 조건	Granite thinking on/off	parser-relaxed Granite native XML
실행 환경	RTX 5070, llama.cpp, Q5_K_M	
주요 관찰 지표	▪ 형식 준수율 (format_ok) ▪ 행동 선택 정확도	▪ 종료 상태 (format_stop/ tool_cap/final) ▪ file/function accuracy

공통 실행 환경에서는 ibm-granite/granite-4.2-8b와 mistralai/Ministral-8B-Instruct-2410을 Q5_K_M 양자화(iMatrix 미사용)로 실행하였다. Context length는 8192로 설정하였으며, 각 문항의 8회 반복에는 반복 번호 0 - 7을 random seed로 사용하였다. 도구 호출 한도는 When2Call 1회, Loc-Bench 8회로 설정하였다.

2.1 실패 지점 분석 기준

형식 실패(format failure)는 응답이 요구된 도구 호출 또는 최종 응답 형식으로 파싱되지 않는 경우다. 행동 선택 실패(action selection failure)는 형식적으로 유효한 응답을 생성했지만 상황에 맞는 행동을 선택하지 못한 경우다. 실행 종료 실패(execution termination failure)는 반복 실행 후에도 호출 한도 내 최종 응답을 제출하지 못한 경우다.

이 구분은 모델 내부의 독립적 기능 모듈이 아니라 실행 로그에서 관찰되는 실패 지점을 의미한다. When2Call의 형식 준수율은 전체 실행 중 format_ok 비율로 산출하며, 행동 선택 정확도는 전체 실행을 분모로 하여 형식적으로 유효하고 정답 행동과 일치한 경우만 정답으로 인정한다. 형식 실패는 분모에 포함하되 오답으로 처리한다. Loc-Bench에서는 종료 상태를 format_stop, tool_cap, final로 구분하며, file/function accuracy는 전체 실행 중 최종 응답의 예측 파일 또는 함수가 정답과 일치한 비율로 산출한다. 그림 1은 세 실패 지점과 이에 대응하는 평가 지표의 관계를 요약한다.

2.2 평가 데이터 및 절차

When2Call은 test/llm_judge 300문항에서 라벨별 UUID를 정렬한 뒤 라운드로빈으로 tool_call 13개, request_for_info 13개, cannot_answer 14개를 선정하였다. 문항 ID는 모델 출력 확인 전에 고정하였으며, 본 연구는 행동 선택과 실패 지점 분석에 초점을 두므로 BFCL 점수는 별도로 산출하지 않았다.

Granite thinking on/off의 행동 선택 정확도 차이에 대한 95% 신뢰구간은 문항 단위 짝지은 부트스트랩 10,000회로 산출하였다. 각 문항의 8회 반복을 먼저 평균한 뒤 조건 간 차이를 복원추출하여, 전체 320회를 독립 표본으로 취급하지 않았다.

Loc-Bench V1 Security Vulnerability는 저장소를 탐색해 수정 대상 파일과 함수를 예측하는 코드 위치 식별 과제다[7]. 원래 29문항 중 fortra__impacket-1636은 호스트 백신의 파일 격리로 저장소를 정상적으로 사용할 수 없어 제외하였다. 모든 조건에서 동일한 28문항을 평가하였다. Function accuracy는 외부 도구 함수 선택이 아니라 예측한 코드 위치의 함수명 일치 여부를 의미한다.

[표 3] Loc-Bench Security 공통 28문항 × 8회 결과 (조건별 N=224, n(%))

조건	format_stop	tool_cap	최종 응답 제출 (final)	File Acc.	Function Acc.
Granite JSON (strict parser)	194 (86.6)	24 (10.7)	6 (2.7)	6 (2.7)	4 (1.8)
Granite JSON (parser-relaxed)	116 (51.8)	87 (38.8)	21 (9.4)	21 (9.4)	17 (7.6)
Granite native XML	57 (25.4)	129 (57.6)	38 (17.0)	34 (15.2)	24 (10.7)
Ministral JSON	34 (15.2)	90 (40.2)	100 (44.6)	36 (16.1)	7 (3.1)

형식 관련 중단 이후의 실행 상태를 확인하기 위해 Granite에 parser-relaxed 조건을 추가하였다. 이 조건은 실패 로그에서 관찰된 JSON 오류 중 숫자 리터럴 뒤에 잘못 추가된 큰따옴표 하나만 제거하며, 일반적인 파서 개선 효과가 아닌 특정 오류 완화 이후의 변화를 확인하기 위한 post-hoc 진단 조건이다. 보조 비교로 Granite의 native XML 형식도 평가하였다. XML은 출력 규약과 도구 목록이 JSON 조건과 다르므로 동일한 개입 효과로 해석하지 않는다. JSON 프롬프트에는 list_dir, read_file, grep, run(target="test")를, native XML에는 앞의 세 탐색 도구를 제시하였다.

III. 결과 및 분석

3.1 높은 형식 준수율에서도 행동 선택 오류가 남는가?

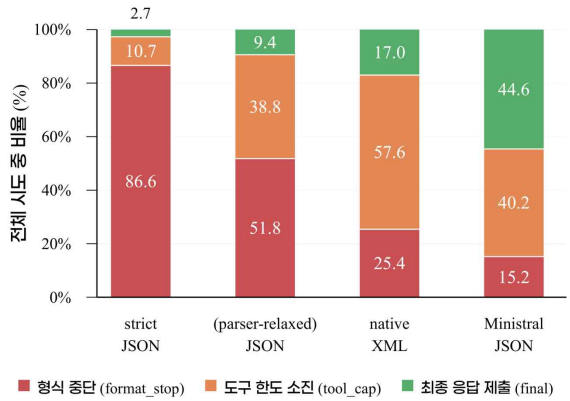
[표 2] When2Call 40문항 × 8회 결과

조건	형식 준수율(%)	행동 선택 정확도(%)
Granite (thinking on)	96.3	62.2
Granite (thinking off)	97.2	65.0
Ministral	99.1	41.3

표 2에서 형식 준수율은 96.3~99.1%였으나 행동 선택 정확도는 41.3~65.0%였다. Granite thinking on에서는 정답이 cannot_answer인 112회 중 43회(38.4%)를 request_for_info로 예측하였고, thinking off에서는 21/112회로 감소하였다. 그러나 전체 행동 선택 정확도 차이(off−on)는 +2.81%p이며, 문항 단위 짝지은 부트스트랩의 95% 신뢰구간 [−9.06, 15.31]은 0을 포함하였다. 따라서 높은 형식 준수율이 높은 행동 선택 정확도를 보장하지 않았으며, thinking 설정 차이는 일관된 정확도 향상보다 오류 구성의 변화로 해석한다.

3.2 형식 관련 중단을 줄이면 반복 실행의 종료 상태는 어떻게 변하는가?

표 3의 format_stop, tool_cap, final은 상호 배타적인 종료 상태이며, final은 최종 응답 제출 여부를 의미한다. Granite JSON에서 parser-relaxed 적용 후 format_stop은 86.6%에서 51.8%로 감소한 반면, tool_cap은 10.7%에서 38.8%, final은 2.7%에서 9.4%로 증가하였다. 이는 형식 중단이 완화되면 일부 실행은 최종 응답까지 진행하지만, 호출 한도 소진과 같은 후속 종료 상태도 드러날 수 있음을 보여준다. 그림 2에서 native XML도 strict JSON 대비 format_stop이 낮고 tool_cap이 높은 방향을 보였으나, 출력 규약과 도구 목록이 다르므로 보조 비교로만 사용한다.



[그림 2] Loc-Bench Security의 종료 상태 분포

문항 단위로 비교하면 strict 대비 parser-relaxed에서 28문항 중 26문항의 형식 중단 횟수가 감소하고 22문항의 한도 소진 횟수가 증가하였다. 8회 모두 final을 제출하지 못한 문항은 parser-relaxed 21개, native XML 18개였으며, 함수 정답을 한 번 이상 제출한 문항은 두 조건 모두 6개였다. 따라서 종료 상태 변화는 일부 문항에만 국한되지 않았지만 함수

정답은 제한된 문항에서 관찰되었다.

Native XML의 한도 소진 129회 중 동일한 도구·인자 조합이 재등장한 실행은 12회(9.3%)였고, 정답 파일에 대한 read_file 호출이 정상 반환되었지만 최종 응답을 제출하지 못한 실행은 64회(49.6%)였다.

대표적으로 Chainlit__chainlit-1441(seed=3)은 read_file(server.py, 830 - 900) → read_file(server.py, 878 - 950) → grep → 추가 read_file 요청 이후 호출 한도로 종료되었다. 다만 재방문은 다른 행 범위나 검색어를 이용한 추가 탐색일 수 있으므로, 현재 로그만으로 탐색 예산 부족과 종료 판단의 문제를 구분할 수는 없다.

IV. 논의 및 한계

본 결과는 최종 정확도와 함께 실패 지점을 분석해야 개선 대상을 구분할 수 있음을 보여준다. 형식 실패가 지배적이면 출력 규약과 파서·도구 인터페이스를, 높은 형식 준수율에도 행동 오답이 많으면 행동 선택 과정을 점검할 필요가 있다. 또한 형식 중단 완화 후 tool_cap과 final이 함께 증가한 결과는 실행 계층의 변경을 최종 정확도뿐 아니라 종료 상태의 변화와 함께 평가할 필요가 있음을 시사한다[4-6, 8, 9].

본 연구는 두 8B 모델, When2Call 40문항, Loc-Bench Security 28문항 및 Q5_K_M/llama.cpp/RTX 5070 환경에 한정된다. 또한 모델별 추론 설정이 다르고 parser-relaxed는 post-hoc 진단 조건이며, Loc-Bench의 호출 한도는 8회로 고정하여 16회 등 한도 증가의 효과를 평가하지 않았다. 일부 중간 모델 출력과 도구 반환 본문도 보존되지 않았다. 따라서 관찰된 실패 비율의 일반화와 tool_cap의 원인 구분에는 한계가 있으며, 호출 한도별 종료 상태·정확도 비교와 모델 확대를 후속 과제로 남긴다.

V. 결론

본 연구는 8B LLM의 도구 사용 실패를 형식 준수, 행동 선택, 실행 종료로 구분하여 분석하였다. When2Call에서는 높은 형식 준수율에도 행동 선택 오류가 남았으며, Loc-Bench Security에서는 형식 중단 완화 후 tool_cap과 final이 함께 증가하였다. 문항별 분석과 실행 로그는 최종 정확도만으로 구분하기 어려운 후속 병목을 보여주지만, 호출 예산 부족과 종료 판단의 원인 구분에는 추가 평가가 필요하다.

Acknowledgement

이 논문은 정부(과학기술정보통신부)의 재원으로 정보통신기획평가원-대학ICT연구센터(ITRC)의 지원을 받아 수행된 연구임(IITP-2026-RS-2020-II201797)

[참고문헌]

[1] Patil, Shishir G., et al. “The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models.” ICML, 2025.

[2] Ross, Hayley, et al. “When2Call: When (not) to Call Tools.” NAACL-HLT, 2025.

[3] Wang, Zhun, et al. “CyberGym: Evaluating AI Agents’ Real-World Cybersecurity Capabilities at Scale.” ICLR, 2026.

[4] Zhu, Pengyu, et al. “A Unified Framework for the Evaluation of LLM Agentic Capabilities.” arXiv:2605.27898, 2026.

[5] Yao, Yilun, et al. “Harness-Bench: Measuring Harness Effects across Models in Realistic Agent Workflows.” arXiv:2605.27922, 2026.

[6] Zhao, Cen Mia, et al. “Beyond Prompts: Measuring and Optimizing LLM Tool-Agent Harnesses.” arXiv:2609.05736, 2026.

[7] Chen, Zhaoling, et al. “LocAgent: Graph-Guided LLM Agents for Code Localization.” ACL, 2025.

[8] Tam, Zhi Rui, et al. “Let Me Speak Freely? A Study on the Impact of Format Restrictions on Performance of Large Language Models.” EMNLP Industry Track, 2024.

[9] Lee, Ivan Yee, et al. “The Format Tax.” arXiv:2604.03616, 2026.